Enrollment No: 240840116014

Step 1:Create sales dataset

```
import pandas as pd
import numpy as np

# Create our own Sales Dataset
data = {
    'OrderID': [
        'ORD101','ORD102','ORD103','ORD104','ORD105',
        'ORD106','ORD107','ORD108','ORD109','ORD110',
        'ORD111','ORD112','ORD113','ORD114','ORD115',
        'ORD116','ORD117','ORD118','ORD119','ORD120'
    ],

    'OrderDate': [
        '2024-01-05','2024-01-12','2024-01-20',
        '2024-02-03','2024-02-15','2024-02-22',
        '2024-03-04','2024-03-13','2024-03-25',
        '2024-04-02','2024-04-14','2024-04-21',
        '2024-05-06','2024-05-17','2024-05-25',
        '2024-06-03','2024-06-12','2024-06-20',
        '2024-07-05','2024-07-15'
    ],

    'Region': [
        'East','West','South','North','East',
        'West','South','North','East','West',
        'South','North','East','West','South',
        'North','East','West','South','North'
    ],

    'Category': [
        'Technology','Furniture','Office Supplies','Technology',
        'Furniture','Office Supplies','Technology','Furniture',
        'Office Supplies','Technology','Furniture','Office Supplies',
        'Technology','Furniture','Office Supplies','Technology',
        'Furniture','Office Supplies','Technology','Furniture'
    ],

    'Product': [
        'Laptop','Chair','Paper','Printer','Table',
        'Pen','Mouse','Desk','Notebook','Monitor',
        'Sofa','File','Keyboard','Chair','Paper',
        'Laptop','Table','Pen','Printer','Desk'
    ],

    'CustomerSegment': [
        'Consumer','Corporate','Small Business','Consumer',
        'Corporate','Consumer','Small Business','Corporate',
        'Consumer','Small Business','Corporate','Consumer',
        'Small Business','Consumer','Corporate','Consumer',
        'Small Business','Corporate','Consumer','Corporate'
    ],

    'Quantity': [
        2,3,10,1,2,20,5,1,15,2,
        1,8,4,2,12,1,3,25,2,1
    ],

    'UnitPrice': [
        50000,3000,100,15000,8000,
```

```
        20,700,12000,80,25000,
        30000,150,1200,3500,100,
        55000,9000,25,14000,15000
    ],

    'Discount': [
        0.05,0.10,0.00,0.05,0.10,
        0.00,0.05,0.10,0.00,0.05,
        0.10,0.00,0.05,0.10,0.00,
        0.05,0.10,0.00,0.05,0.10
    ]
}

# Create DataFrame
sales = pd.DataFrame(data)

# Calculate Sales, Cost and Profit
sales['Sales'] = (
    sales['Quantity'] * sales['UnitPrice'] *
    (1 - sales['Discount'])
).round(2)

sales['Cost'] = (sales['Sales'] * 0.75).round(2)
sales['Profit'] = (sales['Sales'] - sales['Cost']).round(2)

# Introduce missing values for cleaning demonstration
sales.loc[2, 'Region'] = np.nan
sales.loc[5, 'Category'] = np.nan
sales.loc[8, 'Quantity'] = np.nan
sales.loc[12, 'Sales'] = np.nan

# Add one duplicate record
sales = pd.concat([sales, sales.iloc[[0]]], ignore_index=True)

# Save original dataset
sales.to_csv('Sales_Performance_Dataset.csv', index=False)

print("Sales Dataset Created Successfully!")
display(sales.head(10))

print("Dataset Shape:", sales.shape)
```

```
Sales Dataset Created Successfully!
```

	OrderID	OrderDate	Region	Category	Product	CustomerSegment	Quantity	UnitPrice	Discount
0	ORD101	2024-01-05	East	Technology	Laptop	Consumer	2.0	50000	0.05
1	ORD102	2024-01-12	West	Furniture	Chair	Corporate	3.0	3000	0.10
2	ORD103	2024-01-20	NaN	Office Supplies	Paper	Small Business	10.0	100	0.00
3	ORD104	2024-02-03	North	Technology	Printer	Consumer	1.0	15000	0.05
4	ORD105	2024-02-15	East	Furniture	Table	Corporate	2.0	8000	0.10
5	ORD106	2024-02-22	West	NaN	Pen	Consumer	20.0	20	0.00
6	ORD107	2024-03-04	South	Technology	Mouse	Small Business	5.0	700	0.05
7	ORD108	2024-03-13	North	Furniture	Desk	Corporate	1.0	12000	0.10
8	ORD109	2024-03-25	East	Office Supplies	Notebook	Consumer	NaN	80	0.00
9	ORD110	2024-04-02	West	Technology	Monitor	Small Business	2.0	25000	0.05

```
Dataset Shape: (21, 12)
```

Step 2: Load and explore the dataset

```
# Load dataset
sales = pd.read_csv('Sales_Performance_Dataset.csv')

# Display first five records
print("First Five Records:")
display(sales.head())

# Display last five records
print("Last Five Records:")
display(sales.tail())

# Dataset information
print("\nDataset Information:")
sales.info()

# Statistical summary
print("\nStatistical Summary:")
display(sales.describe())

# Check dataset shape
print("\nDataset Shape:", sales.shape)

# Check column names
print("\nColumn Names:")
print(sales.columns.tolist())

# Check missing values
print("\nMissing Values:")
print(sales.isnull().sum())
```

First Five Records:

	OrderID	OrderDate	Region	Category	Product	CustomerSegment	Quantity	UnitPrice	Discount
0	ORD101	2024-01-05	East	Technology	Laptop	Consumer	2.0	50000	0.05
1	ORD102	2024-01-12	West	Furniture	Chair	Corporate	3.0	3000	0.10
2	ORD103	2024-01-20	NaN	Office Supplies	Paper	Small Business	10.0	100	0.00
3	ORD104	2024-02-03	North	Technology	Printer	Consumer	1.0	15000	0.05
4	ORD105	2024-02-15	East	Furniture	Table	Corporate	2.0	8000	0.10

Last Five Records:

	OrderID	OrderDate	Region	Category	Product	CustomerSegment	Quantity	UnitPrice	Discount
16	ORD117	2024-06-12	East	Furniture	Table	Small Business	3.0	9000	0.10
17	ORD118	2024-06-20	West	Office Supplies	Pen	Corporate	25.0	25	0.00

Step 3: Clean and preprocess the dataset

```
# 1. Remove unnecessary columns
sales = sales.loc[:, ~sales.columns.str.contains('^Unnamed')]

# 2. Remove duplicate records
sales = sales.drop_duplicates()

# 3. Convert OrderDate into date format
sales['OrderDate'] = pd.to_datetime(
    sales['OrderDate'], errors='coerce'
)

# 4. Convert numerical columns
numeric_cols = [
    'Quantity', 'UnitPrice', 'Discount',
    'Sales', 'Cost', 'Profit'
]

for col in numeric_cols:
    sales[col] = pd.to_numeric(sales[col], errors='coerce')

# 5. Handle missing numerical values
for col in numeric_cols:
    sales[col] = sales[col].fillna(sales[col].median())

# 6. Handle missing categorical values
categorical_cols = [
    'Region', 'Category', 'Product', 'CustomerSegment'
]

for col in categorical_cols:
    sales[col] = sales[col].fillna(sales[col].mode()[0])

# 7. Remove rows with missing important information
sales = sales.dropna(subset=['OrderID', 'OrderDate'])

# 8. Create additional columns
sales['Year'] = sales['OrderDate'].dt.year
sales['Month'] = sales['OrderDate'].dt.month_name()

sales['ProfitMargin'] = (
    sales['Profit'] / sales['Sales'].replace(0, np.nan)
) * 100
```

```
sales['ProfitMargin'] = sales['ProfitMargin'].fillna(0)

# 9. Display cleaned dataset
print("Cleaned Sales Dataset:")
display(sales.head())

# 10. Check missing values again
print("\nMissing Values After Cleaning:")
print(sales.isnull().sum())

# 11. Save cleaned dataset
sales.to_csv('Cleaned_Sales_Dataset.csv', index=False)

print("\nData Cleaning and Preprocessing Completed!")
```

```
Profit            0
dtype: int64
Cleaned Sales Dataset:
```

	OrderID	OrderDate	Region	Category	Product	CustomerSegment	Quantity	UnitPrice	Discount
0	ORD101	2024-01-05	East	Technology	Laptop	Consumer	2.0	50000	0.05
1	ORD102	2024-01-12	West	Furniture	Chair	Corporate	3.0	3000	0.10
2	ORD103	2024-01-20	East	Office Supplies	Paper	Small Business	10.0	100	0.00
3	ORD104	2024-02-03	North	Technology	Printer	Consumer	1.0	15000	0.05
4	ORD105	2024-02-15	East	Furniture	Table	Corporate	2.0	8000	0.10

```
Missing Values After Cleaning:
OrderID           0
OrderDate         0
Region            0
Category          0
Product           0
CustomerSegment   0
Quantity          0
UnitPrice         0
Discount          0
Sales             0
Cost              0
Profit            0
Year              0
Month             0
ProfitMargin      0
dtype: int64

Data Cleaning and Preprocessing Completed!
```

Step 4: Perform Exploratory Data Analysis (EDA)

```
# 1. Total Sales
print("Total Sales:", sales['Sales'].sum())

# 2. Total Profit
print("Total Profit:", sales['Profit'].sum())

# 3. Total Orders
print("Total Orders:", sales['OrderID'].nunique())

# 4. Total Quantity Sold
```

```
print("Total Quantity:", sales['Quantity'].sum())

# 5. Sales by Region
print("\nSales by Region:")
display(sales.groupby('Region')['Sales'].sum())

# 6. Profit by Category
print("\nProfit by Category:")
display(sales.groupby('Category')['Profit'].sum())

# 7. Sales by Customer Segment
print("\nSales by Customer Segment:")
display(sales.groupby('CustomerSegment')['Sales'].sum())

# 8. Average Profit Margin
print("\nAverage Profit Margin:",
      round(sales['ProfitMargin'].mean(), 2), "%")
```

```
Total Sales: 359750.0
Total Profit: 88377.5
Total Orders: 20
Total Quantity: 107.0

Sales by Region:
```

Region	Sales
East	146700.0
North	92000.0
South	58125.0
West	62925.0

dtype: float64

```
Profit by Category:
```

Category	Profit
Furniture	26200.00
Office Supplies	1306.25
Technology	60871.25

dtype: float64

```
Sales by Customer Segment:
```

CustomerSegment	Sales
Consumer	197200.0
Corporate	75625.0
Small Business	86925.0

dtype: float64

```
Average Profit Margin: 24.28 %
```

Step 5: Visualize EDA results using Plotly

Chart 1: Sales by Region

```
import plotly.express as px

region_sales = sales.groupby('Region', as_index=False)['Sales'].sum()

fig = px.bar(
    region_sales,
    x='Region',
    y='Sales',
    color='Region',
    title='Sales by Region'
)

fig.show()
```

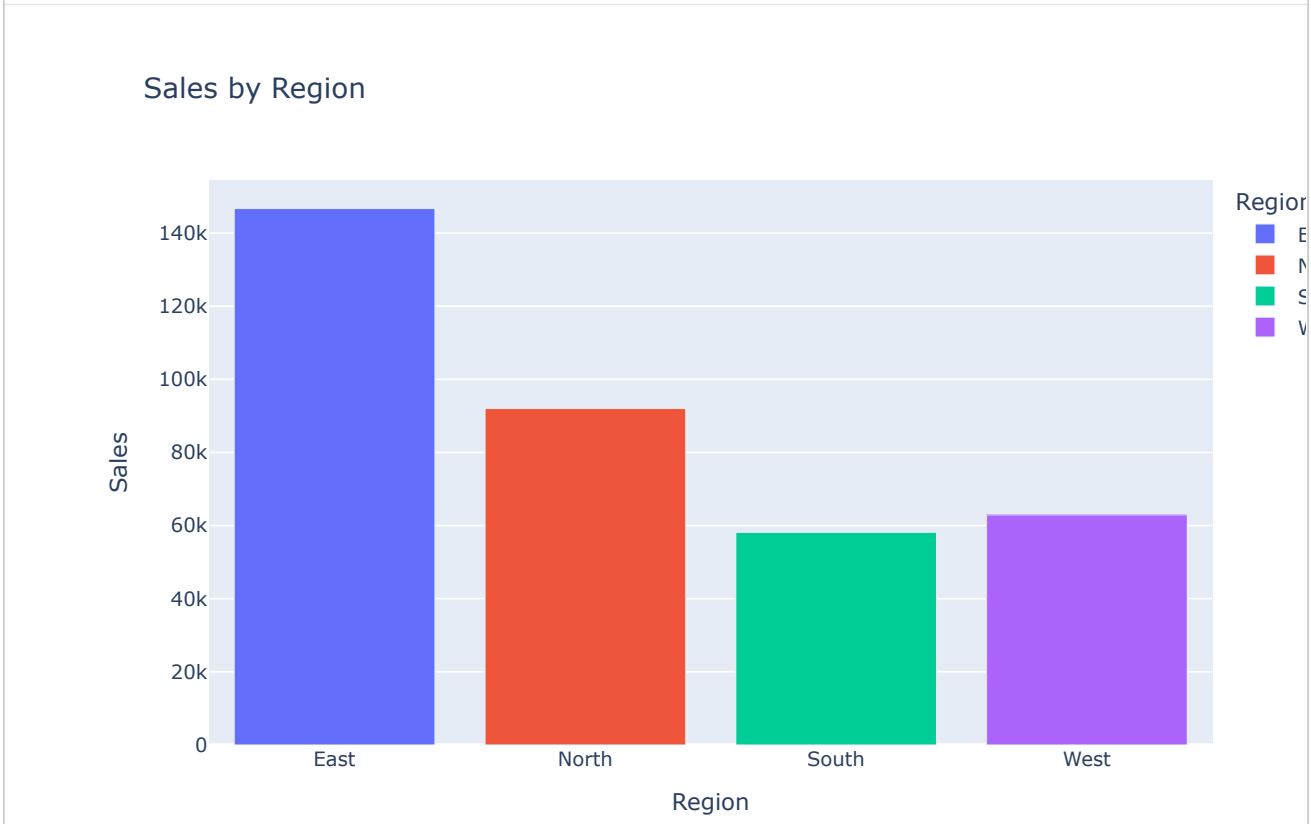


Chart 2: Profit by Category

```
category_profit = sales.groupby(
    'Category', as_index=False
)['Profit'].sum()

fig = px.bar(
    category_profit,
    x='Category',
    y='Profit',
    color='Category',
    title='Profit by Category'
)

fig.show()
```

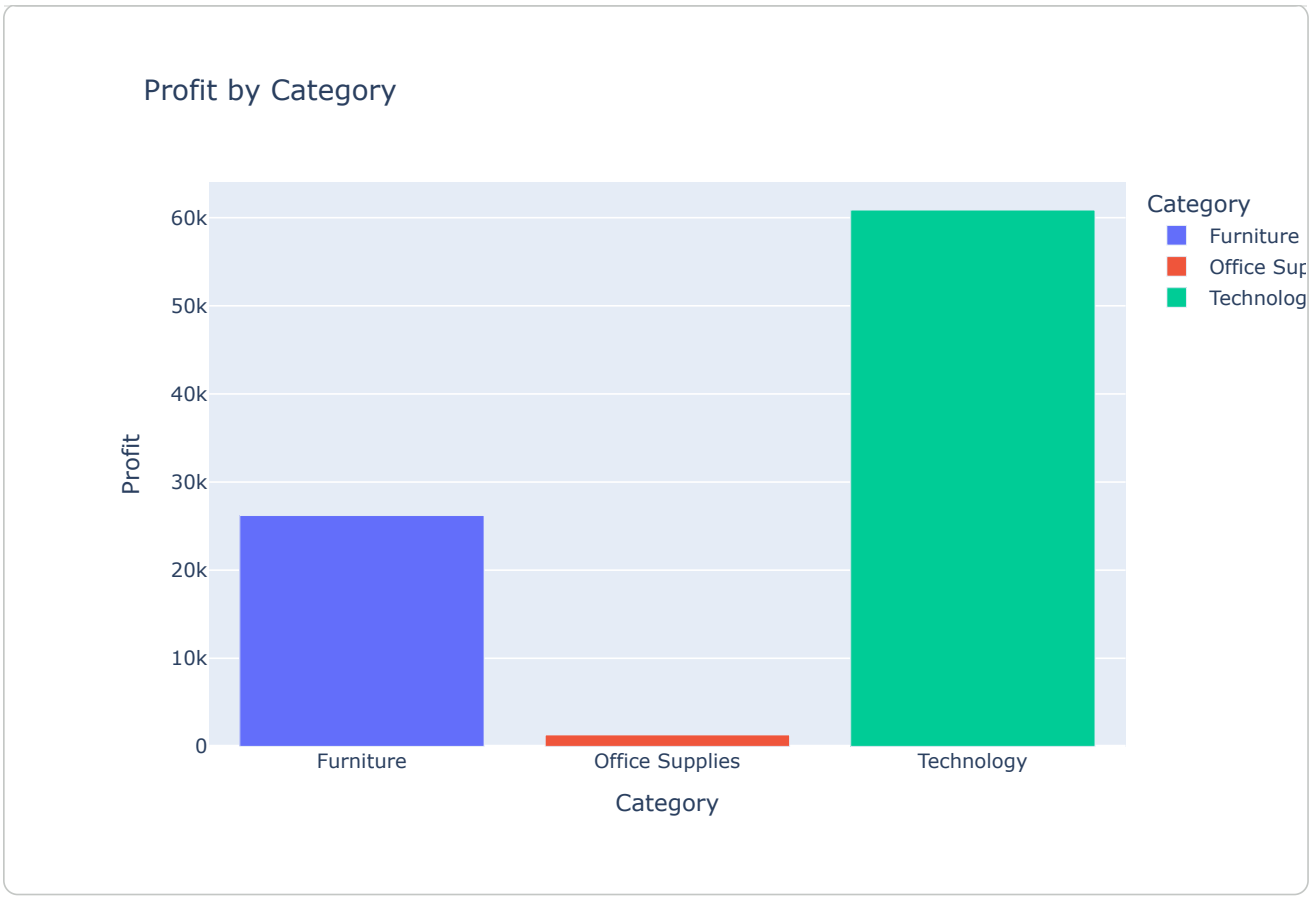


Chart 3: Monthly Sales Trend

```
monthly_sales = sales.groupby(
    'Month', as_index=False
)['Sales'].sum()

month_order = [
    'January', 'February', 'March', 'April',
    'May', 'June', 'July', 'August',
    'September', 'October', 'November', 'December'
]

monthly_sales['Month'] = pd.Categorical(
    monthly_sales['Month'],
    categories=month_order,
    ordered=True
)

monthly_sales = monthly_sales.sort_values('Month')

fig = px.line(
    monthly_sales,
    x='Month',
    y='Sales',
    markers=True,
    title='Monthly Sales Trend'
)

fig.show()
```

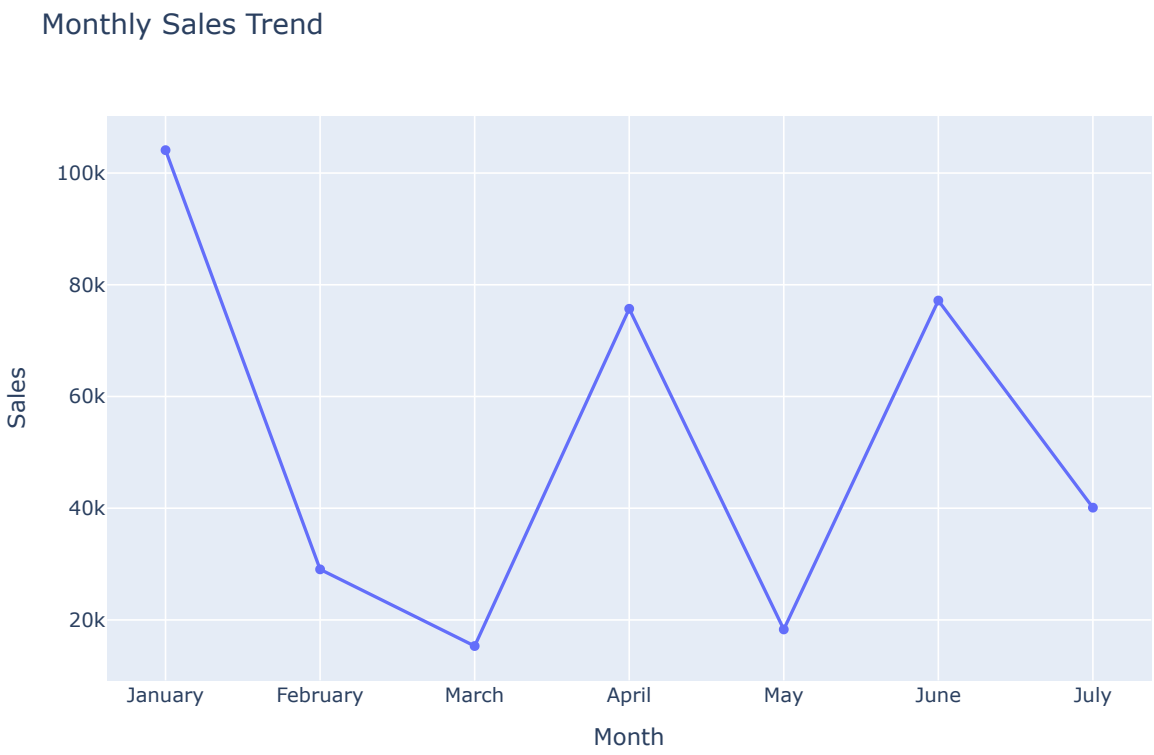
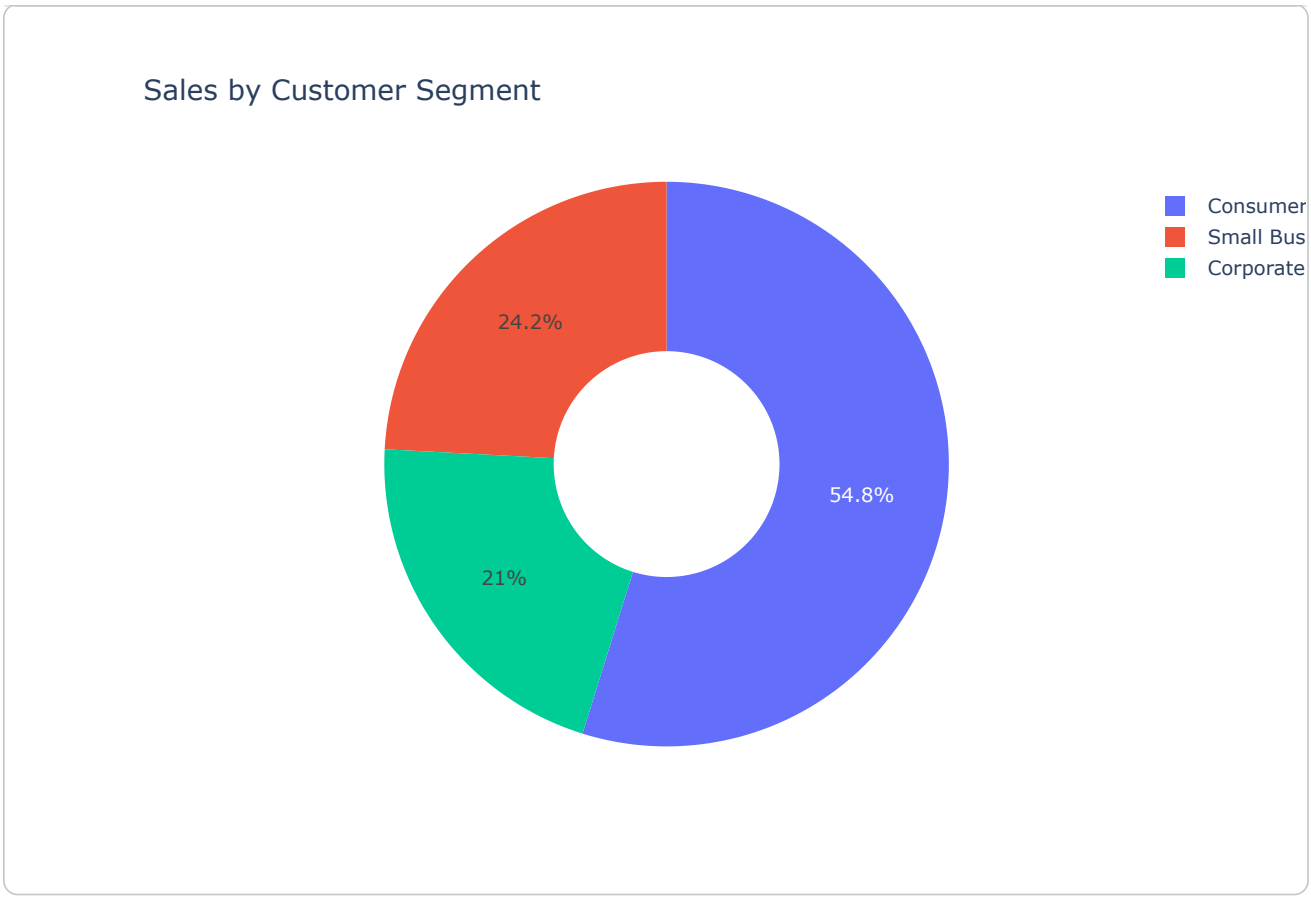


Chart 4: Sales Distribution by Customer Segment

```
segment_sales = sales.groupby(
    'CustomerSegment', as_index=False
)['Sales'].sum()

fig = px.pie(
    segment_sales,
    names='CustomerSegment',
    values='Sales',
    hole=0.4,
    title='Sales by Customer Segment'
)

fig.show()
```



Step 5: Install Dash

```
!pip install -q dash
```

Step 6: Create an interactive dashboard using Dash and Plotly

```
from dash import Dash, dcc, html, Input, Output
import plotly.express as px
import pandas as pd

# Create Dash application
app = Dash(__name__)

app.title = "Sales Performance Dashboard"

# Dropdown options
region_options = [
    {'label': 'All Regions', 'value': 'All'}
] + [
    {'label': r, 'value': r}
    for r in sorted(sales['Region'].unique())
]

category_options = [
    {'label': 'All Categories', 'value': 'All'}
] + [
    {'label': c, 'value': c}
    for c in sorted(sales['Category'].unique())
]

# Dashboard Layout
app.layout = html.Div([
```

```
    html.H1(
        "SALES PERFORMANCE DASHBOARD",
        style={
            'textAlign': 'center',
            'color': '#183153',
            'padding': '15px'
        }
    ),

    html.P(
        "Interactive Business Reporting System",
        style={'textAlign': 'center'}
    ),

    html.Hr(),

    html.Div([
        html.Div([
            html.Label("Select Region"),
            dcc.Dropdown(
                id='region-filter',
                options=region_options,
                value='All',
                clearable=False
            )
        ], style={'width': '48%'}),

        html.Div([
            html.Label("Select Category"),
            dcc.Dropdown(
                id='category-filter',
                options=category_options,
                value='All',
                clearable=False
            )
        ], style={'width': '48%'})
    ], style={
        'display': 'flex',
        'justifyContent': 'space-between',
        'gap': '15px'
    }),

    html.Br(),

    # KPI CARDS
    html.Div([

        html.Div([
            html.H4("Total Sales"),
            html.H2(id='total-sales')
        ], style={
            'backgroundColor': '#DCEEFF',
            'padding': '15px',
            'textAlign': 'center',
            'borderRadius': '10px'
        }),

        html.Div([
            html.H4("Total Profit"),
            html.H2(id='total-profit')
        ], style={
            'backgroundColor': '#DDF5E5',
            'padding': '15px',
            'textAlign': 'center',
            'borderRadius': '10px'
        }),
```

```
        html.Div([
            html.H4("Total Orders"),
            html.H2(id='total-orders')
        ], style={
            'backgroundColor': '#FFF0D9',
            'padding': '15px',
            'textAlign': 'center',
            'borderRadius': '10px'
        }),

        html.Div([
            html.H4("Total Quantity"),
            html.H2(id='total-quantity')
        ], style={
            'backgroundColor': '#F0E4FF',
            'padding': '15px',
            'textAlign': 'center',
            'borderRadius': '10px'
        })

    ], style={
        'display': 'grid',
        'gridTemplateColumns': 'repeat(4, 1fr)',
        'gap': '12px'
    }),

    html.Br(),

    # CHARTS
    dcc.Graph(id='monthly-chart'),

    dcc.Graph(id='region-chart'),

    dcc.Graph(id='category-chart')

], style={
    'fontFamily': 'Arial',
    'padding': '25px',
    'backgroundColor': '#F4F6F9'
})


# Interactive Callback
@app.callback(

    Output('total-sales', 'children'),
    Output('total-profit', 'children'),
    Output('total-orders', 'children'),
    Output('total-quantity', 'children'),

    Output('monthly-chart', 'figure'),
    Output('region-chart', 'figure'),
    Output('category-chart', 'figure'),

    Input('region-filter', 'value'),
    Input('category-filter', 'value')

)

def update_dashboard(selected_region, selected_category):

    # Copy original data
    df = sales.copy()

    # Apply region filter
    if selected_region != 'All':
```

```
    df = df[df['Region'] == selected_region]

# Apply category filter
if selected_category != 'All':
    df = df[df['Category'] == selected_category]

# Calculate KPIs
total_sales = df['Sales'].sum()
total_profit = df['Profit'].sum()
total_orders = df['OrderID'].nunique()
total_quantity = df['Quantity'].sum()

# Monthly Sales Chart
months = [
    'January', 'February', 'March', 'April',
    'May', 'June', 'July', 'August',
    'September', 'October', 'November', 'December'
]

monthly = df.groupby('Month')['Sales'].sum().reindex(
    months, fill_value=0
).reset_index()

fig1 = px.line(
    monthly,
    x='Month',
    y='Sales',
    markers=True,
    title='Monthly Sales Trend'
)

# Profit by Region Chart
region_data = df.groupby(
    'Region', as_index=False
)['Profit'].sum()

fig2 = px.bar(
    region_data,
    x='Region',
    y='Profit',
    color='Region',
    title='Profit by Region'
)

# Sales by Category Chart
category_data = df.groupby(
    'Category', as_index=False
)['Sales'].sum()

fig3 = px.pie(
    category_data,
    names='Category',
    values='Sales',
    hole=0.4,
    title='Sales Distribution by Category'
)

return (
    f"₹{total_sales:,.2f}",
    f"₹{total_profit:,.2f}",
    f"{total_orders}",
    f"{total_quantity:,.0f}",
    fig1,
    fig2,
    fig3
)
```

```
print("Dashboard application created successfully!")
```

```
Dashboard application created successfully!
```

step 7: Run the interactive dashboard

```
# Run dashboard inside Google Colab

app.run(
    jupyter_mode="inline",
    port=8050,
    debug=False
)
```